

Predicción de Fuga de Clientes (Churn) en Telecomunicaciones

Charbel Isaias Chávez Zavala

A01711950

Inteligencia Artificial Avanzada para la Ciencia de Datos I

16 de septiembre de 2026

Resumen

Este trabajo presenta un análisis exploratorio de datos y el desarrollo de un modelo de clasificación binaria para predecir la probabilidad de que un cliente de una empresa de telecomunicaciones abandone el servicio (*churn*).

Se implementó un modelo de regresión logística sin el uso de frameworks modernos. Finalmente, el desempeño del modelo fue evaluado mediante diferentes métricas de clasificación sobre conjuntos de entrenamiento, validación y prueba.

1. Introducción

La inteligencia artificial (IA) es el campo de la ciencia de la computación dedicado a construir sistemas capaces de realizar tareas que, tradicionalmente, requerían inteligencia humana: reconocer patrones, tomar decisiones o hacer predicciones a partir de información incompleta.

Dentro de la IA, el aprendizaje automático (*machine learning*) surge como una alternativa a la programación de reglas explícitas: en lugar de que un experto codifique manualmente el conocimiento necesario para resolver un problema, el sistema aprende dicho conocimiento directamente de los datos. El término fue popularizado por Arthur Samuel en 1959, en el contexto de un programa capaz de aprender a jugar damas mejorando su desempeño con la experiencia, y el campo se consolidó posteriormente con el desarrollo de modelos como el perceptrón de Rosenblatt y, décadas más tarde, con el auge de los métodos estadísticos y del aprendizaje profundo.

Una de las ramas más utilizadas del aprendizaje automático es el **aprendizaje supervisado**, en el cual el modelo se entrena a partir de un conjunto de datos etiquetado. El modelo ajusta sus parámetros internos para aproximar la relación entre las variables de entrada y esa etiqueta, con el objetivo de generalizar correctamente a observaciones nuevas y no vistas durante el entrenamiento. Cuando la etiqueta es una categoría discreta se habla específicamente de un problema de **clasificación**, uno de los dos grandes tipos de tareas de aprendizaje supervisado junto con la regresión (donde la etiqueta es un valor numérico continuo).

1.1. Objetivo

El objetivo de este proyecto es desarrollar un modelo de regresión logística, implementado desde cero utilizando NumPy y Pandas, capaz de clasificar a los clientes de una empresa de telecomunicaciones según su probabilidad de abandonar el servicio (*churn*).

1.2. Descripción del conjunto de datos

El conjunto de datos utilizado es *Telco Customer Churn*. Contiene información de 7,043 clientes de una empresa de telecomunicaciones, descritos mediante 20 variables originales. La variable objetivo es **Churn**, que indica si el cliente abandonó la compañía.

Cuadro 1: Descripción de las variables del dataset Telco Customer Churn

Variable	Descripción
customerID	Identificador único de cada cliente.
gender	Género del cliente. Puede tomar los valores Male o Female.
SeniorCitizen	Indica si el cliente es una persona adulta mayor. 0 representa no y 1 representa sí.
Partner	Indica si el cliente tiene pareja.
Dependents	Indica si el cliente tiene personas dependientes económicamente.
tenure	Número de meses que el cliente ha permanecido con la compañía.
PhoneService	Indica si el cliente tiene contratado el servicio telefónico.
MultipleLines	Indica si el cliente tiene múltiples líneas telefónicas, una sola línea o no cuenta con servicio telefónico.
InternetService	Tipo de servicio de Internet contratado: DSL, fibra óptica o ninguno.
OnlineSecurity	Indica si el cliente tiene contratado el servicio de seguridad en línea.
OnlineBackup	Indica si el cliente tiene contratado el servicio de respaldo en línea.
DeviceProtection	Indica si el cliente cuenta con protección para sus dispositivos.
TechSupport	Indica si el cliente tiene contratado soporte técnico.
StreamingTV	Indica si el cliente tiene contratado un servicio de streaming de televisión.
StreamingMovies	Indica si el cliente tiene contratado un servicio de streaming de películas.
Contract	Tipo de contrato del cliente: mensual, anual o de dos años.

Variable	Descripción
PaperlessBilling	Indica si el cliente utiliza facturación electrónica en lugar de facturación física.
PaymentMethod	Método utilizado por el cliente para realizar sus pagos.
MonthlyCharges	Cantidad cobrada al cliente mensualmente por los servicios contratados.
TotalCharges	Cantidad total cobrada al cliente durante el tiempo que ha permanecido con la compañía.
Churn	Variable objetivo. Indica si el cliente abandonó la compañía. Yes representa abandono y No representa permanencia.

2. Análisis Exploratorio de Datos

El análisis exploratorio de datos (EDA) se realizó sobre un conjunto de datos compuesto por 7,043 clientes y 20 variables originales. El objetivo principal de esta etapa fue evaluar la calidad de los datos, identificar patrones asociados con la cancelación del servicio (*churn*) y determinar las transformaciones necesarias antes de entrenar el modelo de regresión logística.

La variable objetivo es **Churn**, que indica si un cliente abandonó la compañía. Del total de 7,043 clientes, 5,174 permanecieron con el servicio, mientras que 1,869 lo abandonaron. Esto corresponde a una distribución de 73.46 % de clientes sin churn y 26.54 % de clientes con churn.

Esta distribución representa un desbalance moderado entre las clases. Un clasificador que predijera “No Churn” para todos los clientes obtendría aproximadamente 73 % de exactitud sin haber aprendido ninguna relación entre las variables. Por esta razón, el desempeño del modelo no se evaluará únicamente mediante accuracy, sino también mediante precision, recall y F1-score.

Los hallazgos del EDA determinaron las principales decisiones utilizadas para preparar los datos para el modelo. En primer lugar, **customerID** fue eliminado debido a su naturaleza de identificador. Posteriormente, **TotalCharges** fue convertido explícitamente a formato numérico y los 11 valores faltantes asociados con clientes con **tenure** = 0 fueron reemplazados por cero.

La variable objetivo **Churn** fue transformada de valores categóricos a una representación binaria, donde 0 representa permanencia y 1 representa abandono. Para las variables categóricas binarias se utilizó una codificación binaria mediante una sola variable, mientras que las variables con tres o más categorías fueron transformadas mediante *one-hot encoding*. De esta manera, se evita imponer un orden numérico artificial entre categorías que no poseen una relación ordinal.

2.1. Visualizaciones

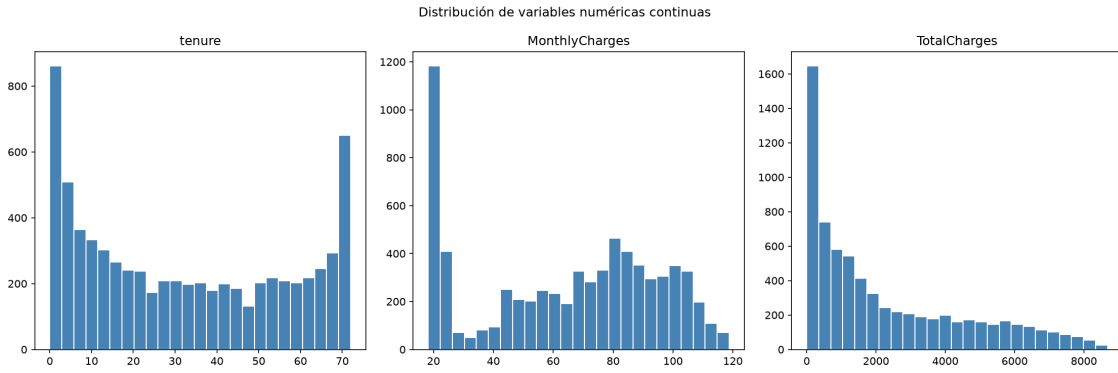


Figura 1: Distribución de las variables numéricas continuas (`tenure`, `MonthlyCharges`, `TotalCharges`).

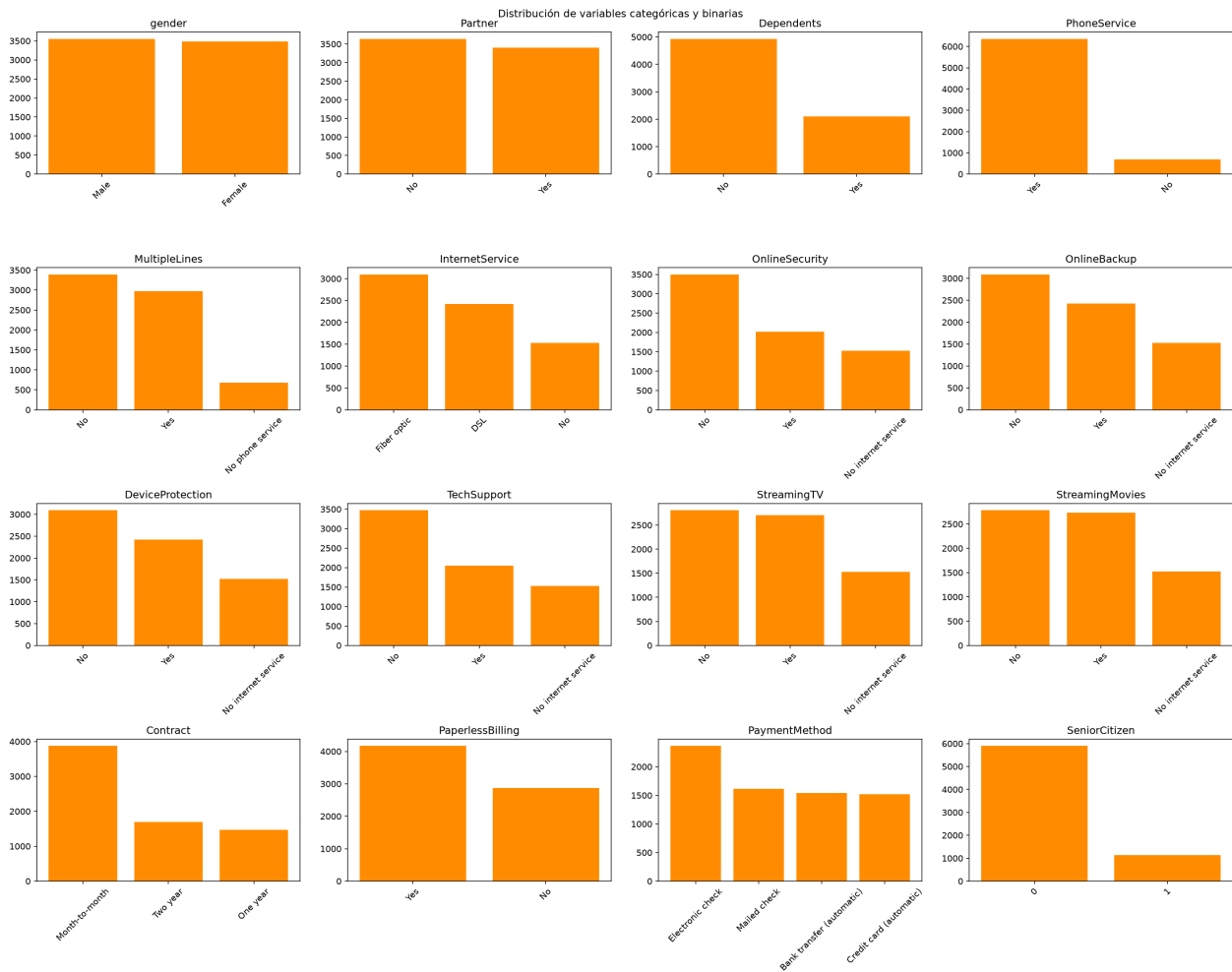


Figura 2: Distribución de las variables categóricas y binarias del conjunto de datos.

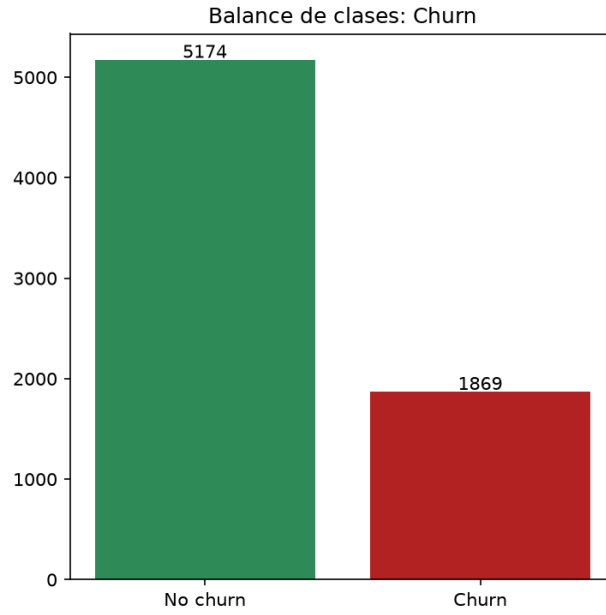


Figura 3: Balance de clases de la variable objetivo **Churn**.

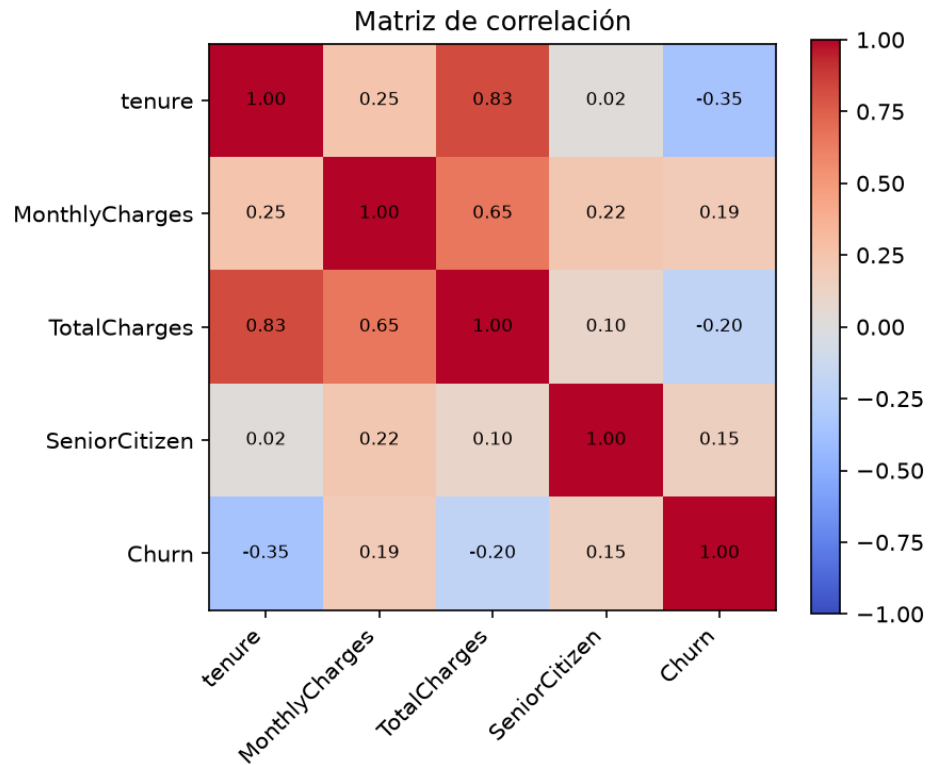


Figura 4: Matriz de correlación entre las variables numéricas y **Churn**.

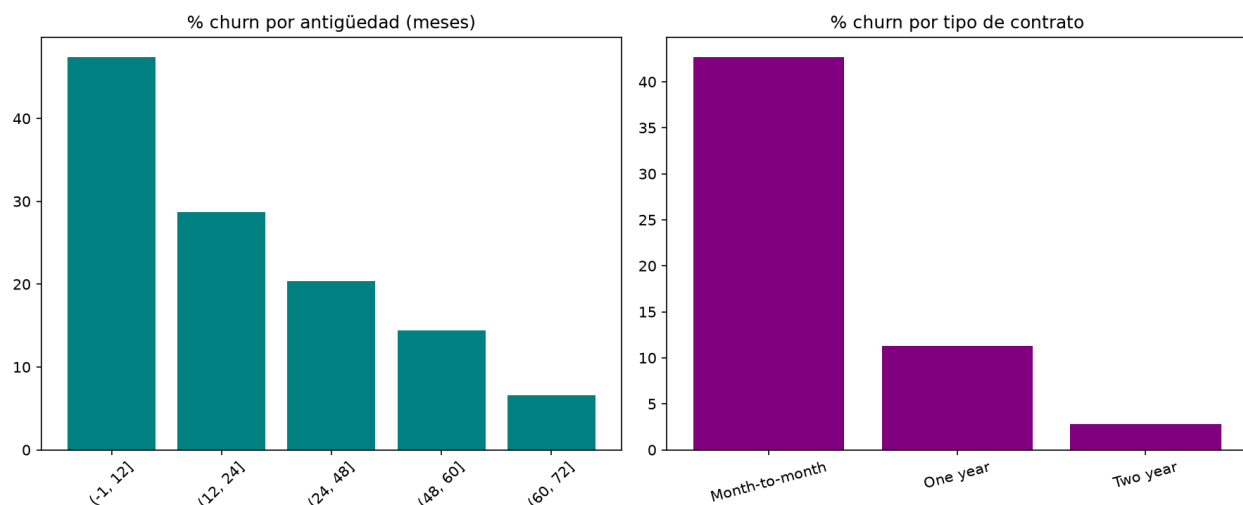


Figura 5: Tasa de churn (%) por rango de antigüedad y por tipo de contrato.

3. Preprocesamiento de Datos

En esta sección se describen las transformaciones realizadas antes del entrenamiento del modelo. El preprocesamiento se implementó en dos etapas: la limpieza y codificación del conjunto de datos (`telco_eda.py`), que produce el archivo `telco_churn_processed.csv`, y la estandarización y partición de los datos (`telco_modelo_manual.py`), realizada inmediatamente antes del entrenamiento.

3.1. Tratamiento de valores faltantes

El único valor faltante detectado en el conjunto de datos corresponde a la variable `TotalCharges`, con 11 registros nulos (0.16 % del total). Dado que esta columna se almacena como texto en el archivo original, primero se realizó una conversión explícita a tipo numérico mediante `pd.to_numeric(..., errors="coerce")`; los valores no convertibles quedaron como `NaN`, lo que permitió identificarlos.

Al analizar estos 11 registros se confirmó que todos corresponden a clientes con `tenure = 0`, es decir, clientes recién dados de alta que aún no han generado ningún cargo. Por lo tanto, el valor faltante representa en realidad un cero válido de negocio y no un dato perdido por error.

Método utilizado: imputación con el valor fijo 0.

```
df["TotalCharges"] = df["TotalCharges"].fillna(0)
```

Se descartó imputar con la media o la mediana, ya que esto asignaría un cargo acumulado considerable (aprox. 2280, la media de la columna) a clientes con cero meses de antigüedad, introduciendo un sesgo artificial en los datos. El resto de las 19 columnas no presenta valores nulos.

3.2. Codificación de variables categóricas

El conjunto de datos contiene 15 variables categóricas (además de la variable objetivo `Churn`), codificadas mediante dos estrategias distintas según su cardinalidad:

- **Mapeo binario manual (0/1)**, para columnas con exactamente dos categorías: `gender`, `Partner`, `Dependents`, `PhoneService` y `PaperlessBilling`. Con solo dos categorías, una única columna binaria representa toda la información sin redundancia.
- **One-hot encoding**, para columnas con tres o más categorías: `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`, `Contract` y `PaymentMethod`, mediante `pd.get_dummies`. Se prefirió esta estrategia sobre una codificación ordinal (0, 1, 2, ...) porque estas categorías no poseen un orden numérico natural; una codificación ordinal induciría al modelo a interpretar relaciones de magnitud inexistentes entre las categorías.

La variable objetivo `Churn` se codificó de forma independiente como 0/1 mediante `(df["Churn"] == "Yes").astype(int)`. Como resultado de este proceso, el conjunto de datos pasó de 20 a 41 columnas, quedando íntegramente en formato numérico.

3.3. Normalización o estandarización

Las variables numéricas fueron estandarizadas (*z-score*) antes de entrenar el modelo:

```
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)
sigma[sigma == 0] = 1.0

X_train = (X_train - mu) / sigma
X_val    = (X_val - mu) / sigma
X_test   = (X_test - mu) / sigma
```

La media (`mu`) y la desviación estándar (`sigma`) de cada variable se calculan exclusivamente sobre el conjunto de entrenamiento, y esas mismas estadísticas se reutilizan para transformar validación y prueba, evitando así una fuga de información (*data leakage*) del conjunto de evaluación hacia el ajuste del modelo.

Esta transformación es necesaria porque las variables numéricas originales presentan escalas muy distintas. Sin estandarizar, el descenso de gradiente avanzaría de forma desigual entre variables de distinta escala, ralentizando o desestabilizando la convergencia. Adicionalmente, al dejar todas las variables en una escala comparable, los coeficientes finales del modelo pueden interpretarse directamente como una medida de importancia relativa de cada variable.

3.4. División de los datos

El conjunto de datos se dividió en tres subconjuntos (entrenamiento, validación y prueba), en lugar de solo dos, con el fin de monitorear el comportamiento del modelo durante el entrenamiento sin utilizar el conjunto de prueba antes de la evaluación final:

- 60 % para entrenamiento (train).
- 20 % para validación (validation).
- 20 % para prueba (test).

La partición se realizó mediante una permutación aleatoria de los índices de fila (con semilla fija `np.random.seed(42)` para garantizar la reproducibilidad de los resultados), repartida posteriormente en los tres tramos indicados. Con el conjunto de datos completo (7043 registros), esta partición resulta en aproximadamente 4227 registros de entrenamiento, 1408 de validación y 1408 de prueba.

El conjunto de **entrenamiento** se utiliza para ajustar los pesos del modelo mediante descenso de gradiente; el de **validación** permite calcular la función de pérdida en cada época sobre datos no vistos durante el ajuste, con el fin de detectar sobreajuste; y el de **prueba** se reserva exclusivamente para la evaluación final del modelo ya entrenado, simulando su desempeño esperado sobre datos nuevos.

4. Modelo de Regresión Logística

En esta sección se describe el primer modelo de clasificación implementado: una regresión logística para clasificación binaria, programada sin el uso de frameworks de ML modernos. En la Sección 6 se presenta un segundo modelo, un Random Forest implementado con `scikit-learn`, con fines comparativos.

4.1. Descripción del modelo

El modelo calcula, para cada cliente, una combinación lineal de sus variables de entrada (ya estandarizadas) ponderadas por un vector de pesos w (uno por cada una de las 40 variables del conjunto de datos codificado) más un término de intercepto (*bias*) b :

$$z = Xw + b \quad (1)$$

En la implementación, esta operación se realiza de forma vectorizada para todos los clientes simultáneamente mediante el operador de multiplicación matricial de NumPy (`X @ w + b`).

La salida z se transforma mediante la función sigmoide, que la comprime al intervalo $(0, 1)$, interpretándose como la probabilidad estimada de que el cliente abandone el servicio:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

La clasificación final se obtiene aplicando un umbral de decisión de 0.5 sobre esta probabilidad: si $\sigma(z) \geq 0.5$ se predice `Churn = 1`; en caso contrario, se predice `Churn = 0`.

4.2. Función de pérdida

Para clasificación binaria se utilizó la función (*binary cross-entropy*), que penaliza con mayor intensidad las predicciones incorrectas realizadas con alta confianza:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3)$$

En la implementación se añade una constante $\epsilon = 10^{-12}$ dentro de cada logaritmo para evitar indeterminaciones numéricas ($\log(0)$) cuando $\hat{y}_i$ se aproxima a 0 o a 1.

4.3. Optimización

Los pesos y el *bias* se ajustan mediante descenso de gradiente (*gradient descent*): en cada época se calcula la predicción sobre todo el conjunto de entrenamiento y se actualizan los parámetros en la dirección que reduce la pérdida:

$$w := w - \alpha \frac{\partial L}{\partial w} \quad (4)$$

donde α representa la tasa de aprendizaje (*learning rate*). El entrenamiento se ejecutó con $\alpha = 0,1$ durante 5000 épocas, valores que resultaron en una convergencia estable de la función de pérdida, como se observa en la Figura 6.

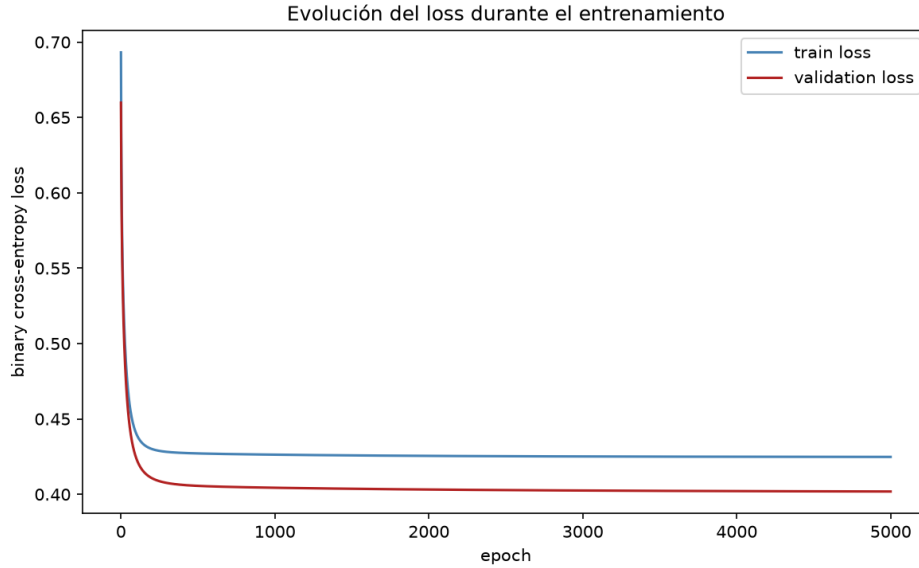


Figura 6: Evolución de la función de pérdida (entropía cruzada binaria) sobre los conjuntos de entrenamiento y validación a lo largo de las épocas.

Ambas curvas descienden de forma conjunta y se estabilizan sin separarse, lo cual indica que el modelo converge adecuadamente y no presenta señales evidentes de sobreajuste con los hiperparámetros utilizados.

5. Resultados

En esta sección se presentan los resultados obtenidos después del entrenamiento del modelo, evaluados sobre los tres subconjuntos de datos (entrenamiento, validación y prueba).

5.1. Métricas de evaluación

Dado el desbalance de clases identificado en el análisis exploratorio (73.46 % *No Churn* / 26.54 % *Churn*), el desempeño del modelo se evaluó mediante accuracy, precision, recall y F1-score, calculados de forma manual a partir de la matriz de confusión de cada subconjunto.

Cuadro 2: Métricas de evaluación del modelo por conjunto de datos.

Conjunto	Accuracy	Precision	Recall	F1-Score
Entrenamiento	0.7991	0.6487	0.5277	0.5820
Validación	0.8097	0.6698	0.5703	0.6160
Prueba	0.8217	0.6885	0.5941	0.6378

Las métricas se mantienen consistentes entre los tres conjuntos, sin una caída notable en validación o prueba respecto a entrenamiento, lo que confirma la ausencia de sobreajuste observada previamente en la curva de pérdida (Figura 6).

5.2. Matriz de confusión

Se presenta la matriz de confusión, calculada sobre el conjunto de prueba, para analizar el comportamiento del modelo.

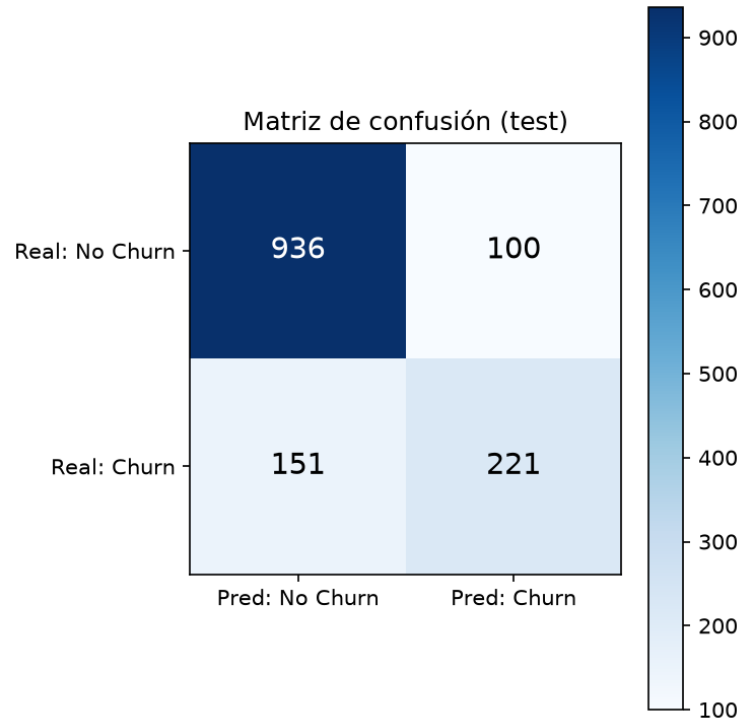


Figura 7: Matriz de confusión del modelo sobre el conjunto de prueba.

Sobre los 1408 clientes del conjunto de prueba, el modelo clasificó correctamente a 936 de los 1036 clientes que efectivamente permanecieron (verdaderos negativos) y a 221 de los 372 clientes que efectivamente abandonaron el servicio (verdaderos positivos). Se registraron 100 falsos positivos (clientes que el modelo predijo como abandono sin serlo) y 151 falsos negativos (clientes que abandonaron y el modelo no logró identificar).

5.3. Interpretación de los resultados

El modelo alcanza una exactitud del 82.17% sobre el conjunto de prueba, un valor considerablemente superior al 73.46% que obtendría un clasificador trivial que siempre prediga *No Churn*, lo que confirma que el modelo efectivamente capturó relaciones útiles entre las variables y el abandono de clientes.

El recall (59.41%) es notablemente menor que la precision (68.85%), lo que indica que el modelo es más conservador para etiquetar a un cliente como riesgo de abandono: deja pasar una proporción considerable de los clientes que sí abandonan (falsos negativos) a cambio de cometer relativamente pocas falsas alarmas (falsos positivos).

Al estar las variables estandarizadas, los coeficientes del modelo son comparables entre sí y permiten interpretar la importancia relativa de cada variable. Las variables con mayor influencia coinciden con los hallazgos del análisis exploratorio: **tenure** (antigüedad) presenta el coeficiente de mayor magnitud, con signo negativo (a mayor antigüedad, menor probabilidad de churn), seguido de **TotalCharges** (positivo) y de las variables de tipo de contrato: **Contract_Two year** reduce la probabilidad de churn, mientras que **Contract_Month-to-month**

la incrementa. `InternetService_Fiber optic` también aparece entre las variables con mayor efecto positivo sobre el churn, consistente con la alta tasa de abandono observada para ese segmento en el EDA (Figura 5).

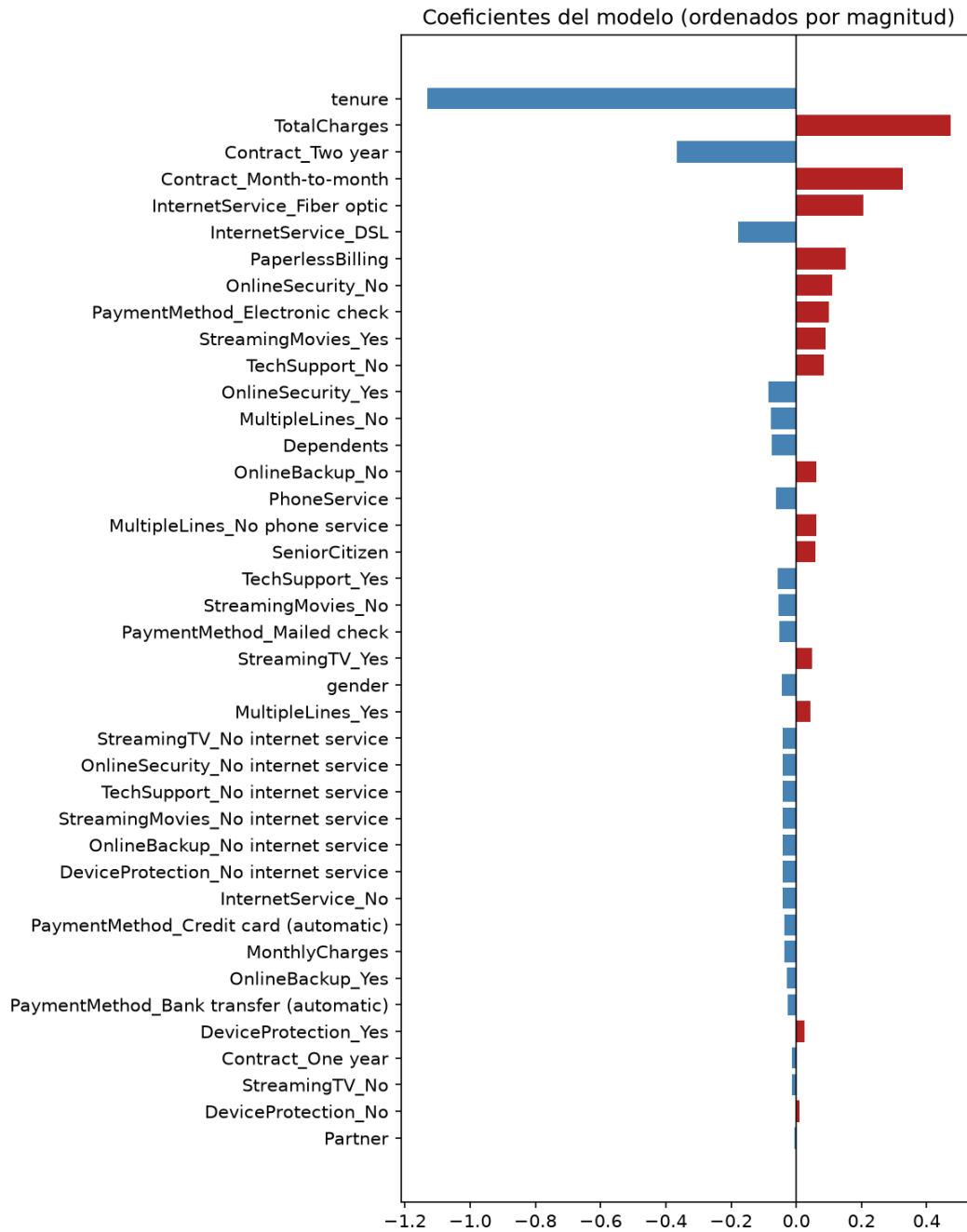


Figura 8: Coeficientes del modelo de regresión logística, ordenados por magnitud.

6. Random Forest

Como segundo modelo, y con fines comparativos, se implementó un **Random Forest** utilizando la biblioteca scikit-learn (`sklearn.ensemble.RandomForestClassifier`). A diferencia de la regresión logística de la Sección 4, un Random Forest es un modelo de *ensamble* compuesto por múltiples árboles de decisión, cada uno entrenado sobre una muestra aleatoria con reemplazo (*bootstrap*) del conjunto de entrenamiento y considerando, en cada división, un subconjunto aleatorio de variables. La predicción final se obtiene por votación mayoritaria entre todos los árboles del bosque, lo que le permite capturar relaciones no lineales e interacciones entre variables que un modelo estrictamente lineal no puede representar.

6.1. Descripción del modelo y preprocesamiento

Se utilizó la misma partición de datos que en el modelo de regresión logística (60 % entrenamiento, 20 % validación, 20 % prueba, con semilla fija para reproducibilidad), obtenida en este caso mediante `train_test_split` de scikit-learn. A diferencia de la regresión logística, las variables numéricas **no** fueron estandarizadas antes de entrenar el Random Forest: los árboles de decisión dividen cada variable mediante umbrales (por ejemplo, `tenure ≤ 12`), por lo que son invariantes a la escala de las variables y no requieren este paso de preprocesamiento.

El modelo se configuró con los siguientes hiperparámetros:

- `n_estimators = 300`: número de árboles en el bosque.
- `max_depth = 10`: profundidad máxima de cada árbol, para limitar el sobreajuste individual de cada uno.
- `min_samples_leaf = 5`: número mínimo de muestras requeridas en una hoja, otra medida de control de sobreajuste.
- `criterion = "entropy"`: los árboles seleccionan cada división maximizando la ganancia de información (reducción de entropía) en lugar del índice Gini (la alternativa por defecto en scikit-learn); en la práctica, ambos criterios produjeron resultados prácticamente idénticos sobre este conjunto de datos.

La Figura 9 muestra el primer árbol del bosque, limitando la visualización a sus primeros tres niveles para mantenerla legible (el árbol completo entrenado tiene una profundidad máxima de diez niveles).

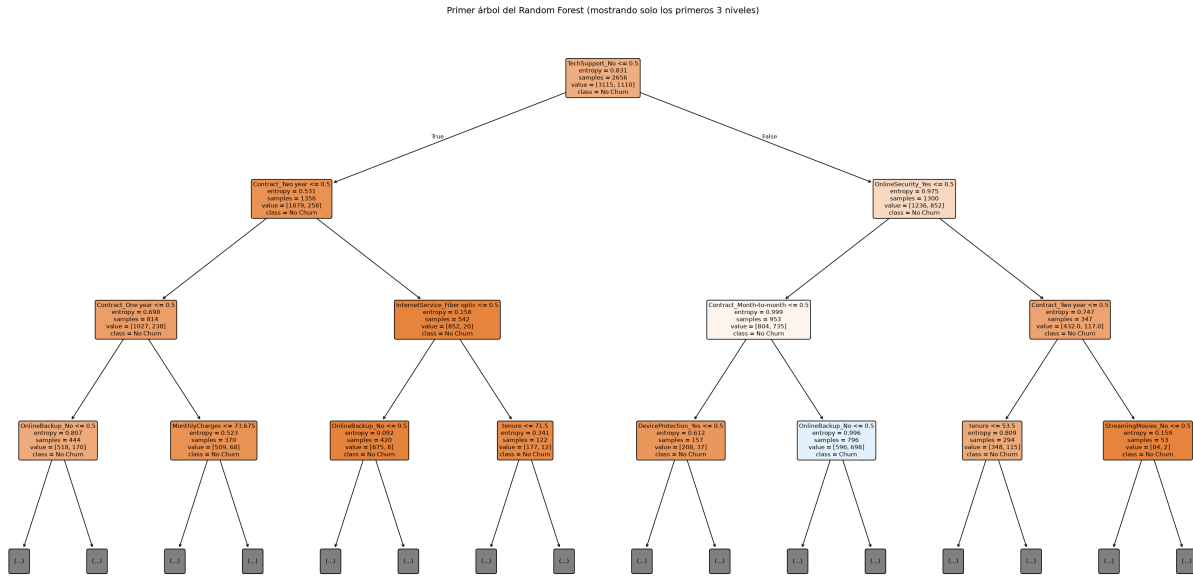


Figura 9: Primer árbol del Random Forest, mostrando únicamente los primeros tres niveles de división.

6.2. Resultados

Cuadro 3: Métricas de evaluación del Random Forest por conjunto de datos.

Conjunto	Accuracy	Precision	Recall	F1-Score
Entrenamiento	0.8379	0.7586	0.5696	0.6507
Validación	0.8070	0.6732	0.5450	0.6023
Prueba	0.8169	0.7085	0.5175	0.5981

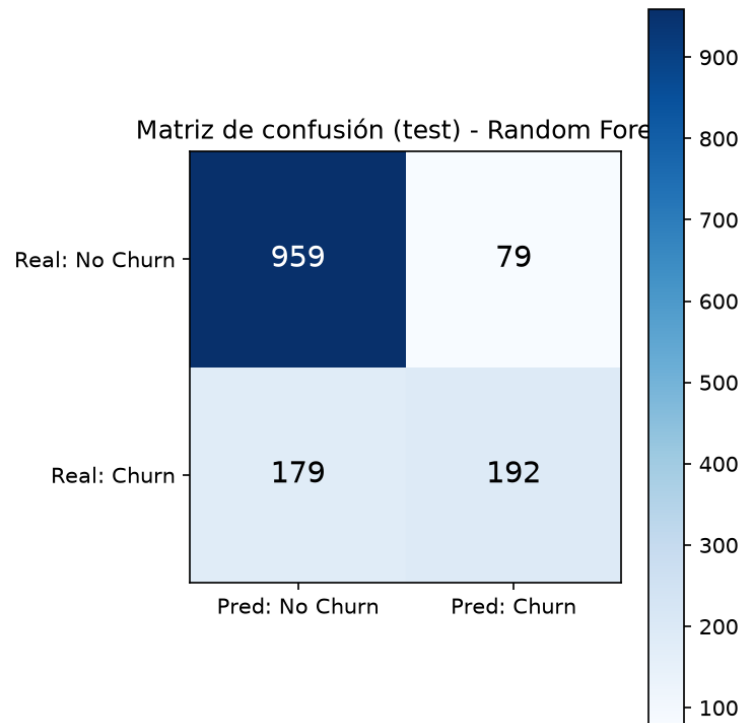


Figura 10: Matriz de confusión del Random Forest sobre el conjunto de prueba.

Sobre los 1409 clientes del conjunto de prueba, el modelo clasificó correctamente a 959 de los 1038 clientes que efectivamente permanecieron (verdaderos negativos) y a 192 de los 371 clientes que efectivamente abandonaron el servicio (verdaderos positivos), con 79 falsos positivos y 179 falsos negativos.

La importancia de cada variable, calculada como la reducción promedio de impureza que aporta en todos los árboles del bosque, se muestra en la Figura 11.

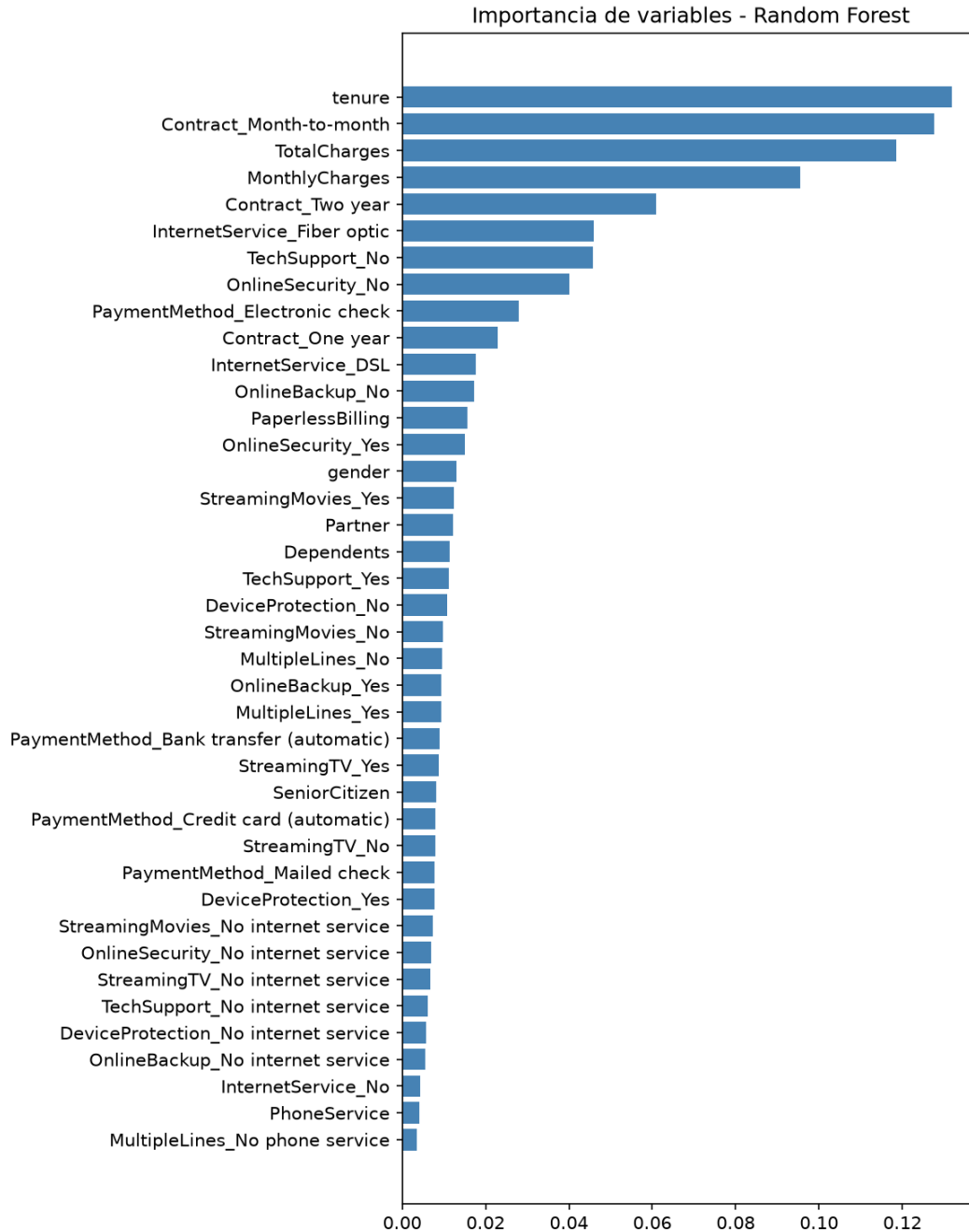


Figura 11: Importancia de las variables en el Random Forest, ordenadas por magnitud.

6.3. Interpretación y comparación con la regresión logística

Las variables más importantes para el Random Forest —*tenure*, *Contract_Month-to-month*, *TotalCharges*, *MonthlyCharges* y *Contract_Two year*— coinciden en gran medida con las de mayor coeficiente en la regresión logística (Figura 8), lo que refuerza la validez de ambos modelos: llegan a conclusiones similares sobre qué factores impulsan el abandono de clientes, a pesar de tener mecanismos internos completamente distintos.

Cuadro 4: Comparación de métricas sobre el conjunto de prueba entre ambos modelos.

Modelo	Accuracy	Precision	Recall	F1-Score
Regresión Logística	0.8217	0.6885	0.5941	0.6378
Random Forest	0.8169	0.7085	0.5175	0.5981

Ambos modelos alcanzan una exactitud muy similar (en torno al 82 %), pero con un comportamiento distinto frente al desbalance de clases: el Random Forest logra mayor precisión (70.85 % frente a 68.85 %) pero un recall notablemente menor (51.75 % frente a 59.41 %), lo que se traduce en un F1-score inferior. En otras palabras, el Random Forest es aún más conservador que la regresión logística para etiquetar a un cliente como riesgo de abandono, dejando pasar una mayor proporción de clientes que sí abandonan. Adicionalmente, la brecha entre las métricas de entrenamiento (83.79 % accuracy) y las de prueba (81.69 %) es más amplia que la observada en la regresión logística, lo que sugiere un mayor grado de sobreajuste pese a las restricciones aplicadas (`max_depth` y `min_samples_leaf`).

6.4. Ponderación de clases (`class_weight`)

El conjunto de datos está desbalanceado (73.46 % de clientes que no abandonan frente a 26.54 % que sí lo hacen), y el Random Forest base entrenado sin ningún ajuste por este desbalance resulta demasiado conservador: obtiene una precisión razonable (70.85 %) pero un recall bajo (51.75 %), es decir, deja sin detectar a casi la mitad de los clientes que efectivamente abandonan. Como esto es costoso desde el punto de vista de negocio (un cliente en riesgo que no se detecta no puede recibir ninguna acción de retención), se implementó un nuevo script, `telco_framework_class_weight.py`, que entrena, con los mismos datos, la misma partición y los mismos hiperparámetros de árbol que el modelo base, tres variantes:

- `class_weight=None`: línea base, idéntica a `telco_framework.py`.
- `class_weight="balanced"`: los pesos de cada clase se fijan de forma inversamente proporcional a su frecuencia, iguales para todo el bosque.
- `class_weight="balanced_subsample"`: los pesos se recalculan en cada árbol según la muestra *bootstrap* específica con la que se entrenó ese árbol.

Cuadro 5: Comparación de métricas sobre el conjunto de prueba entre las tres variantes de `class_weight`.

Configuración	Accuracy	Precision	Recall	F1-Score
Sin ponderar (None)	0.8169	0.7085	0.5175	0.5981
<code>balanced</code>	0.7658	0.5346	0.8544	0.6577
<code>balanced_subsample</code>	0.7814	0.5566	0.8356	0.6681

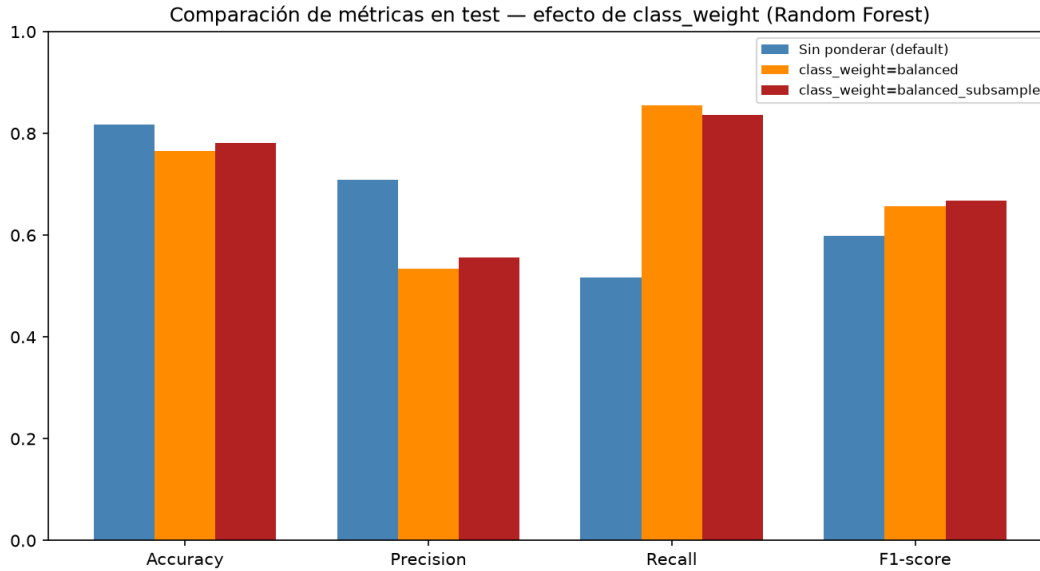


Figura 12: Comparación de accuracy, precision, recall y F1-score sobre el conjunto de prueba entre las tres configuraciones de `class_weight`.

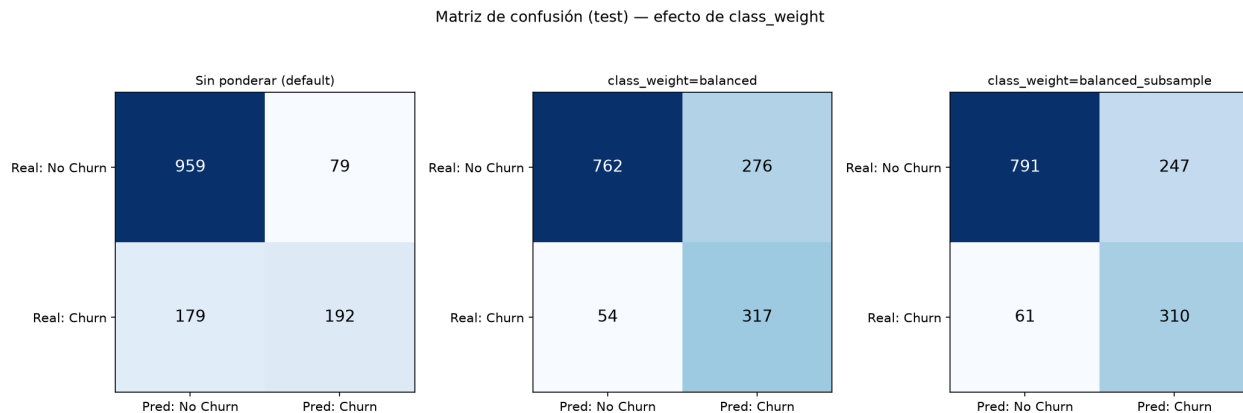


Figura 13: Matrices de confusión sobre el conjunto de prueba: sin ponderar vs. `class_weight="balanced"` vs. `class_weight="balanced_subsample"`.

Ponderar las clases tiene un efecto marcado y en la dirección esperada: el recall sube de 51.75 % a 85.44 % con `balanced` (y a 83.56 % con `balanced_subsample`), es decir, el modelo pasa de detectar a poco más de la mitad de los clientes que abandonan a detectar a más de 8 de cada 10. Esto se traduce en una mejora del F1-score, de 0.5981 (sin ponderar) a 0.6681 (`balanced_subsample`). El costo de esta mejora es una caída en precision (de 70.85 % a 53.46–55.66 %) y en accuracy (de 81.69 % a 76.58–78.14 %): al bajar el umbral implícito de decisión hacia la clase minoritaria, el modelo también aumenta considerablemente los falsos positivos (de 79 a 247–276 sobre 1409 clientes de prueba).

Entre las dos variantes ponderadas, `balanced_subsample` obtiene un mejor equilibrio que `balanced` en las tres métricas relevantes (accuracy, precision y F1-score), a la vez que man-

tiene un recall comparable (83.56 % frente a 85.44 %), lo que la convierte en la configuración preferible si se busca maximizar F1-score.